

System Architecture & Technical Flowcharts

This document provides a comprehensive visual and structural breakdown of the **AIFOD Daily AI News Aggregator**, detailing infrastructure, multi-tier RSS scraping with anti-blocking proxies, Google Gemini AI reasoning & filtering, and Gmail API dispatch.

1. High-Level Infrastructure & System Overview

```
flowchart TD
    %% -----
    %% Trigger & Orchestration Layer
    %% -----
    subgraph TriggerLayer ["1. Trigger & Scheduling Layer"]
        CRON["🕒 GCP Cloud Scheduler"]
        ai-news-aggregator-daily-trigger["ai-news-aggregator-daily-trigger"]
        ai-news-aggregator-daily-trigger --> CLI["🖥️ Local CLI / Manual Trigger"]
        CLI --> python["python -m src.main --hours 24"]
        python --> SA["🔑 IAM Service Account"]
        ai-news-scheduler-sa["ai-news-scheduler-sa"]
        ai-news-scheduler-sa --> roles["(roles/run.invoker + OIDC)"]
    end

    %% -----
    %% Compute & Hosting Infrastructure
    %% -----
    subgraph ComputeLayer ["2. Compute & Runtime Infrastructure (GCP Cloud Run)"]
        CR["🌀 Cloud Run Service"]
        ai-news-aggregator-krjp["ai-news-aggregator-krjp"]
        ai-news-aggregator-krjp --> region["(Region: us-central1)"]
        region --> GUNICORN["🐘 Gunicorn WSGI Server"]
        GUNICORN --> workers["(1 Worker, 8 Threads, Port: 8080)"]
        workers --> FLASK["🍷 Flask App"]
        FLASK --> src_app_py["src/app.py"]
        src_app_py --> post["(POST /?hours=24)"]
        post --> MAIN["⚙️ Orchestration Core"]
        MAIN --> src_main_py["src/main.py (main())"]
    end

    %% -----
    %% Ingestion & Anti-Blocking Resilience Layer
    %% -----
    subgraph ScrapingLayer ["3. Ingestion & Multi-Tier Resilience Layer"]
        src_rss_parser_py["(src/rss_parser.py)"]
        src_rss_parser_py --> QUERIES["🔍 Search Query Matrix"]
        QUERIES --> queries["• 18 KR Queries (AI, ODA, 격차, 정책)  
• 18 JP Queries (AI, ODA, 格差, 政策)"]
        queries --> POOL["⚡ ThreadPoolExecutor"]
        POOL --> pool_config["(max_workers=15 concurrent tasks)"]
    end
```

```

    subgraph FallbackChain ["Egress IP Anti-Blocking Chain"]
        DIRECT["❶ Direct Google News RSS
Rotating User-Agents (Chrome, Safari, Firefox)"]
        PROXY1["❷ Primary CORS Proxy
corsproxy.io/?url=..."]
        PROXY2["❸ Secondary CORS Proxy
api.allorigins.win/raw?url=..."]
    end

    GN_KR["KR Google News RSS (KR)
hl=ko&gl=KR&ceid=KR:ko"]
    GN_JP["JP Google News RSS (JP)
hl=ja&gl=JP&ceid=JP:ja"]

    DEDUP["✂ In-Memory Deduplication & Time Filter
• URL Hash Set (seen_urls)
• Publication Window (<= 24h)
• Top 100 Date-Sorted Buffer"]
    end

    %% -----
    %% AI Intelligence Layer
    %% -----
    subgraph AILayer ["4. AI Reasoning & Enrichment Layer (src/llm_filter.py)"]
        GEMINI["🔮 Google Gemini API
Model: gemini-2.5-flash-lite
(Auto-Retry & Rate-Limit Backoff)"]
        PROMPT["🔄 Multi-Stage Prompt Pipeline
1. AIFOD Mission Alignment
2. Zero-Tolerance Deduplication
3. Thematic Diversity Filtering
4. Natural English Translation
5. Top 5 Selection & Ranking"]
        ENRICH["📄 Structured JSON Extraction
• English Title & 2-3 Sentence Summary
• Relevance Explanation
• AIFOD Strategic Insight
• Practitioner Discussion Q&A"]
    end

    %% -----
    %% Email Generation & Delivery Layer
    %% -----
    subgraph DeliveryLayer ["5. Email Formatting & Delivery
(src/email_sender.py)"]
        HTML_BUILDER["🎨 Premium Responsive HTML Builder
• Plus Jakarta Sans Typography
• KR/JP Country Badges
• Gradient Header & KPI Counters"]
        GMAIL_AUTH["🔑 Gmail OAuth 2.0 Auth
credentials.json + token.json
(Auto-refresh Token Flow)"]
        GMAIL_API["📧 Google Gmail API

```

```

(users.messages.send)"]
    INBOX["📧 Recipient Mailbox
henry.hyunwookim@gmail.com"]
end

%% -----
%% CI/CD & Deployment Layer
%% -----
subgraph DevOpsLayer ["6. DevOps & Build Infrastructure"]
    DOCKER["🐳 Dockerfile (python:3.11-slim)"]
    CB["🏗️ Google Cloud Build"]
    AR["📦 Artifact Registry"]
    PS["📄 PowerShell Deploy Script
deployment/deploy_cloud.ps1"]
end

%% Flow Connections
CRON -->|OIDC Auth POST| SA --> CR
CLI --> MAIN

CR --> GUNICORN --> FLASK --> MAIN

MAIN --> POOL
QUERIES --> POOL

POOL --> DIRECT
DIRECT -->|On 403 / 429 / 503 Block| PROXY1
PROXY1 -->|On Failure| PROXY2

DIRECT --> GN_KR & GN_JP
PROXY1 --> GN_KR & GN_JP
PROXY2 --> GN_KR & GN_JP

GN_KR & GN_JP --> DEDUP
DEDUP --> GEMINI

GEMINI --> PROMPT --> ENRICH

ENRICH --> MAIN
MAIN --> HTML_BUILDER

GMAIL_AUTH --> GMAIL_API
HTML_BUILDER --> GMAIL_API
GMAIL_API --> INBOX

PS --> CB --> AR --> CR
DOCKER --> CB

```

2. Detailed AI Pipeline: Google Gemini Processing & Value-Add Generation

This flowchart illustrates the step-by-step data transformation, prompt engineering constraints, retry resilience, and structured JSON parsing within `[llm_filter.py]`(file:///d:/OneDrive/GitHub/AI-news-aggregator-KRJP/src/llm_filter.py).

```

flowchart TD
    %% Input Articles
    subgraph InputStage ["1. Pre-Processing & Token Management"]
        RAW_ARTICLES["📄 Raw Candidate Articles (Up to 100)"]
        MINIMIZER["✂️ Payload Optimization"]
    end
    Extract only: index, title, source, country, query, snippet[:300]
    COMPACT_JSON["📦 Compact JSON Payload"]
    end

    %% Prompt Architecture
    subgraph PromptStage ["2. Expert Prompt Architecture (AIFOD News Analyst)"]
        P_MISSION["🎯 5 Core AIFOD Mission Themes:
        1. Bridging the AI Gap (Digital Divide)
        2. AI Governance & Social Equity
        3. Capacity Building & Education
        4. International Cooperation & ODA (KOICA/JICA)
        5. AI for Social Good (Health, Agriculture, Climate)"]

        P_RULES["🛡️ Cognitive Filtering & Deduplication Rules:
        • Zero Tolerance for Identical Press Releases
        • Group Substantially Overlapping Trends
        • Thematic Redundancy Filter (Max 1 per theme type)
        • Cross-Lingual KR/JP Event Deduplication
        • Select Top 5 Most Impactful Developments"]

        P_SCHEMA["📋 Enforced JSON Schema Output:
        • english_title
        • english_summary (2-3 sentences)
        • relevance_explanation
        • aifod_insight
        • aifod_question
        • aifod_suggested_answer"]
    end

    %% Execution & Resilience Engine
    subgraph ExecutionStage ["3. Gemini API Invocation & Resilience Loop"]
        CALL_GEMINI["⚡ Call Google Gemini API
        gemini-2.5-flash-lite
        genai.GenerativeModel.generate_content()"]

        subgraph ErrorHandler ["Retry & Rate-Limit Engine (Max 5 Attempts)"]
            CHECK_RESP["API Call
            Success?"]
            RATE_LIMIT["Error Type?"]
            WAIT_429["⌚ 429 / Resource Exhausted:
            Sleep 60 seconds"]
            WAIT_EXP["⌚ Other Error:
            Exponential Backoff (3s * attempt)"]
        end
    end

```

```

        RETRY["🔄 Retry Attempt (1..5)"]
        FAIL_SAFE["⚠️ Fallback: Return Empty List"]
    end
end

%% JSON Extraction & Synthesis
subgraph ExtractionStage ["4. Response Parsing & Metadata Synthesis"]
    RAW_TEXT["📄 Raw LLM Output Text"]
    STRIP_MD["✂️ Markdown Cleaner"]
    Strip ``json fences & Regex match { ... }"
    JSON_PARSE["🔍 JSON Deserializer (json.loads)"]
    MERGE_META["🔗 Re-attach Original Metadata:
original_title, link, source, country, published"]
    FINAL_TOP5["🏆 Final Top 5 AIFOD Briefings"]
end

%% Connections
RAW_ARTICLES --> MINIMIZER --> COMPACT_JSON
COMPACT_JSON & P_MISSION & P_RULES & P_SCHEMA --> CALL_GEMINI

CALL_GEMINI --> CHECK_RESP
CHECK_RESP -- "❌ Failure" --> RATE_LIMIT
RATE_LIMIT -- "HTTP 429" --> WAIT_429 --> RETRY --> CALL_GEMINI
RATE_LIMIT -- "Other Error" --> WAIT_EXP --> RETRY --> CALL_GEMINI
RETRY -- "Exceeded 5 Retries" --> FAIL_SAFE

CHECK_RESP -- "✅ Success" --> RAW_TEXT
RAW_TEXT --> STRIP_MD --> JSON_PARSE --> MERGE_META --> FINAL_TOP5

```

3. Detailed Data Ingestion: Anti-Blocking RSS API Fallback Pipeline

Google Cloud Run datacenter egress IPs often encounter HTTP 503 rate-limiting from Google News RSS endpoints. The multi-tiered fallback pipeline in [\[rss_parser.py\]](file:///d:/OneDrive/GitHub/AI-news-aggregator-KRJP/src/rss_parser.py) guarantees resilient data gathering:

```

flowchart TD
    %% Query Generation
    subgraph QueryDispatch ["1. Query Formulation & Concurrent Dispatch"]
        K_LIST["KR 18 Korean Keywords  
(AI 개발도상국, 국제협력, ODA, 정보격차...)"]
        J_LIST["JP 18 Japanese Keywords  
(AI 途上国, 国際協力, ODA, デジタル格差...)"]
        DISPATCH["⚡ ThreadPoolExecutor (15 concurrent workers)"]
        Execute["get_feed_articles(query, country)"]
    end

    %% Tier 1 Direct
    subgraph Tier1 ["2. Tier 1: Direct HTTP Request"]
        UA_ROTATE["🖥️ Desktop User-Agent Pool  
(Chrome 120 / Safari 17 / Firefox 121)"]
    end

```

```

    REQ_DIRECT["🌐 Direct GET Request to
news.google.com/rss/search?q=...
(Timeout: 10s)"]
    CHECK_T1{"Response
Valid?"}
    end

%% Tier 2 Primary Proxy
    subgraph Tier2 ["3. Tier 2: Primary CORS Proxy Fallback"]
        REQ_PROXY1["🕒 Fallback: Route through corsproxy.io
https://corsproxy.io/?
(Timeout: 12s)"]
        CHECK_T2{"Proxy 1
Success?"}
    end

%% Tier 3 Secondary Proxy
    subgraph Tier3 ["4. Tier 3: Secondary CORS Proxy Fallback"]
        REQ_PROXY2["🕒 Fallback: Route through allorigins.win
https://api.allorigins.win/raw?url=
(Timeout: 12s)"]
        CHECK_T3{"Proxy 2
Success?"}
    end

%% Feed Parser & Deduplication
    subgraph IngestionOutput ["5. Feed Parsing, Filtering & In-Memory
Deduplication"]
        FEED_PARSER["📄 feedparser.parse(xml_data)"]
        TIME_FILTER["🕒 Time Window Filter: pub_date >= (now - hours_back)"]
        SEEN_CHECK{"URL already in
seen_urls hash set?"}
        ADD_UNIQUE["➕ Append to unique_articles list
Add URL to seen_urls"]
        DISCARD["🗑 Discard Duplicate"]
        SORT_BUFF["📊 Sort by Publication Date Descending"]
    end

%% Flow Connections
    K_LIST & J_LIST --> DISPATCH
    DISPATCH --> UA_ROTATE --> REQ_DIRECT --> CHECK_T1

    CHECK_T1 -- "✅ 200 OK & Has Entries" --> FEED_PARSER
    CHECK_T1 -- "❌ 403 / 429 / 503 / Bozo" --> REQ_PROXY1 --> CHECK_T2

    CHECK_T2 -- "✅ Success" --> FEED_PARSER
    CHECK_T2 -- "❌ Failed" --> REQ_PROXY2 --> CHECK_T3

    CHECK_T3 -- "✅ Success" --> FEED_PARSER
    CHECK_T3 -- "❌ Failed" --> DISCARD

    FEED_PARSER --> TIME_FILTER --> SEEN_CHECK
    SEEN_CHECK -- "No (New)" --> ADD_UNIQUE --> SORT_BUFF
    SEEN_CHECK -- "Yes (Duplicate)" --> DISCARD

```

4. End-to-End API Sequence Diagram

This sequence diagram illustrates the temporal interactions between Cloud Scheduler, Cloud Run, external RSS endpoints, Google Gemini, and the Gmail API:

```
sequenceDiagram
    autonumber
    actor Scheduler as  GCP Cloud Scheduler
    participant CloudRun as  Cloud Run (Flask / Gunicorn)
    participant Orchestrator as  main.py
    participant RSSParser as  rss_parser.py
    participant GoogleNews as  Google News RSS / Proxies
    participant Gemini as  Google Gemini API (2.5-flash-lite)
    participant EmailSender as  email_sender.py
    participant GmailAPI as  Gmail REST API
    actor Recipient as  User Inbox

    %% Step 1: Scheduling
    Scheduler->>CloudRun: POST /?hours=24 (with OIDC Bearer Token)
    activate CloudRun
    CloudRun->>Orchestrator: main(hours_back=24)
    activate Orchestrator

    %% Step 2: Auth Check
    Orchestrator->>EmailSender: Verify / Refresh Gmail OAuth Token
    EmailSender-->>Orchestrator: Credentials Valid (token.json)

    %% Step 3: RSS Scraping
    Orchestrator->>RSSParser: parse_and_filter_articles(hours_back=24)
    activate RSSParser
    par Concurrent Fetch (15 Workers)
        RSSParser->>GoogleNews: Direct GET with Desktop UA
        GoogleNews-->>RSSParser: 200 OK / 503 Blocked
        opt On 503 Block
            RSSParser->>GoogleNews: Fallback via corsproxy.io / allorigins.win
            GoogleNews-->>RSSParser: 200 OK (Proxied XML)
        end
    end
    deactivate RSSParser
    RSSParser->>RSSParser: URL Deduplication & Timestamp Sorting
    RSSParser-->>Orchestrator: List of Unique Articles (~30-80 articles)
    deactivate RSSParser

    %% Step 4: AI Reasoning
    Orchestrator->>Gemini: generate_content(prompt + articles_json)
    activate Gemini
    Note over Gemini: 1. AIFOD Mission Alignment
    2. Zero-Tolerance Deduplication
    3. Thematic Diversity Ranking
    4. English Translation
```

5. Insight & Q&A Synthesis

Gemini-->>Orchestrator: Structured JSON (Top 5 Selected Articles)

deactivate Gemini

%% Step 5: Email Assembly & Dispatch

Orchestrator-->>EmailSender: send_digest_email(top_5, total_fetched, date_str)

activate EmailSender

EmailSender-->>EmailSender: Compile Responsive HTML with Plus Jakarta Sans &

Country Badges

EmailSender-->>GmailAPI: users.messages.send(raw=MIME_Base64)

activate GmailAPI

GmailAPI-->>Recipient: Deliver Daily Digest Email

GmailAPI-->>EmailSender: Message ID (Success)

deactivate GmailAPI

EmailSender-->>Orchestrator: Email Dispatched Successfully

deactivate EmailSender

%% Step 6: Completion

Orchestrator-->>CloudRun: Execution Stats (success=True, counts)

deactivate Orchestrator

CloudRun-->>Scheduler: 200 OK (JSON Response)

deactivate CloudRun